

CIARLO Emilio Luca

CLARET Nathan

FABRE Lucas

RESTAURATION D'IMAGES



2022-2023

M.El-Mahbouby - M.Couffignal

1	INTRODUCTION	2
2	FORMALISATION THÉORIQUE	3
2.1	Question 1	3
2.2	Question 2	5
2.2.1	a)	5
2.2.2	b)	5
2.2.3	c)	6
2.2.4	d)	7
3	RÉSOLUTION NUMÉRIQUE	8
3.1	Question 1	8
3.2	Question 2	8
3.3	Question 3	9
3.4	Question 4	9
3.5	Question 5	10
3.6	Question 6	11
3.7	Question 7	12
3.7.1	a) masque(F,v)	12
3.7.2	b) D(r)	12
3.7.3	c) A(X,M)	13
3.7.4	d) GCM(M,F,X0,epsilon)	13
3.7.5	e) restauration(F,M,epsilon)	14
3.7.6	Difficultés en python	15
4	Conclusion	20

CHAPTER 1

INTRODUCTION

L'un des défis les plus importants de l'imagerie numérique est la restauration d'image, cela consiste à améliorer la qualité d'une image en éliminant les défauts et les perturbations qui peuvent altérer sa clarté. En effet, les images peuvent être dégradées par différents facteurs. Les restaurer permet de récupérer des informations précieuses contenues dans ces dernières et peut avoir des applications pratiques dans de nombreux domaines. Les ingénieurs, en particulier, jouent un rôle fondamental dans le développement et l'application de techniques efficaces de restauration d'image afin de résoudre des problèmes complexes et répondre à des besoins spécifiques dans diverses industries. À travers ce rapport, nous allons vous montrer les différentes étapes théoriques et numériques qui permettent la restauration d'images.

CHAPTER 2

FORMALISATION THÉORIQUE

La première partie de notre travail consistera à établir un cadre formel pour le problème de restauration d'image. Ce cadre formel nous permettra de considérer les images comme des fonctions à deux variables et de discrétiser les équations résultantes.

2.1 Question 1

En nous inspirant de la démonstration des équations d'Euler-Lagrange on va démontrer que les points critiques de S vérifient l'équation suivante:

$$\frac{dL}{du} = \frac{d}{dx} \frac{dL}{du_1} + \frac{d}{dy} \frac{dL}{du_2}$$

Pour cela on utilisera la dérivé directionnelle de S par rapport à v .

$$D_v S(u) = \lim_{\lambda \rightarrow 0} \frac{S(u + \lambda v) - S(u)}{\lambda}$$

$$\begin{aligned} \frac{S(u + \lambda v) - S(u)}{\lambda} &= \iint_{\Omega} L(x, y, u(x, y) + \lambda v(x, y), u_1(x, y) + \lambda v_1(x, y), u_2(x, y) + \lambda v_2(x, y)) \\ &\quad - L(x, y, u(x, y), u_1(x, y), u_2(x, y)) \, dx \, dy \end{aligned}$$

Or, grâce à un developpement de Taylor à l'ordre 1 on a:

$$\begin{aligned} L(x, y, u(x, y) + \lambda v(x, y), u_1(x, y), u_2(x, y)) \, dx \, dy &= L(x, y, u(x, y), u_1(x, y), u_2(x, y)) + \\ &\quad \nabla L(x, y, u, u_1, u_2) \begin{pmatrix} 0 \\ 0 \\ \lambda v \\ \lambda v_1 \\ \lambda v_2 \end{pmatrix} + \lambda \sqrt{v^2 + v_1^2 + v_2^2} \epsilon(v, v_1, v_2) \, dx \, dy \end{aligned}$$

Avec:

$$\lim_{\lambda \rightarrow 0} \epsilon(\lambda) = 0$$

Donc: $\lambda\sqrt{v^2 + v_1^2 + v_2^2}\epsilon(v, v_1, v_2) = 0$

Ainsi:

$$S(u + \lambda v) - S(u) = \iint_{\Omega} L(x, y, u(x, y), u_1(x, y), u_2(x, y)) + \nabla L(x, y, u, u_1, u_2) \begin{pmatrix} 0 \\ 0 \\ \lambda v \\ \lambda v_1 \\ \lambda v_2 \end{pmatrix} \\ - L(x, y, u(x, y), u_1(x, y), u_2(x, y)) dx dy$$

$$= \iint_{\Omega} \nabla L^T(x, y, u, u_1, u_2) \begin{pmatrix} 0 \\ 0 \\ \lambda v \\ \lambda v_1 \\ \lambda v_2 \end{pmatrix} dx dy = \iint_{\Omega} \lambda \nabla L(x, y, u, u_1, u_2) \begin{pmatrix} 0 \\ 0 \\ v \\ v_1 \\ v_2 \end{pmatrix} dx dy$$

$$D_v L(x, y, v, v_1, v_2) = \lim_{\lambda \rightarrow 0} \frac{S(x, y, u + \lambda v, u_1 + \lambda v_1, u_2 + \lambda v_2) - S(x, y, u, u_1, u_2)}{\lambda}$$

Enfin:

$$D_v L(x, y, v, v_1, v_2) = \lim_{\lambda \rightarrow 0} \frac{\iint_{\Omega} \lambda \nabla L(x, y, u, u_1, u_2) \begin{pmatrix} 0 \\ 0 \\ v \\ v_1 \\ v_2 \end{pmatrix} dx dy}{\lambda}$$

$$= \iint_{\Omega} \nabla L^T(x, y, u, u_1, u_2) \begin{pmatrix} 0 \\ 0 \\ v \\ v_1 \\ v_2 \end{pmatrix} dx dy$$

$$D_v S(u) = \iint_{\Omega} \left(\frac{dL}{du} v + \frac{dL}{du_1} v_1 + \frac{dL}{du_2} v_2 \right) dx dy$$

$$D_v S(u) = \iint_{\Omega} \sum_{i=0}^2 \frac{dL}{du_i} v_i dx dy = \iint_{\Omega} \left(\frac{dL}{du} v + \frac{dL}{du_1} \frac{dv}{dx} + \frac{dL}{du_2} \frac{dv}{dy} \right) dx dy$$

On utilise une intégrale par partie:

$$\iint_{\Omega} \frac{dL}{du_1} \frac{dv}{dx} dx dy = \iint_{\Omega} \frac{dL}{du_1} v dl - \iint_{\Omega} \frac{dL}{du_1} \frac{d}{dx} v dx dy$$

$$\iint_{\Omega} \frac{dL}{du_2} \frac{dv}{dy} dx dy = \iint_{\Omega} \frac{dL}{du_2} v dl - \iint_{\Omega} \frac{dL}{du_2} \frac{d}{dy} v dx dy$$

u est un point critique de $S \rightarrow$ alors $D_v S(u) = 0$, pour tout v.

$$\Leftrightarrow \iint_{\Omega} \frac{dL}{du} v - \frac{d}{dx} \frac{dL}{du_1} v - \frac{d}{dy} \frac{dL}{du_2} v dx dy = 0$$

$$\Leftrightarrow \frac{dL}{du} = \frac{d}{dx} \frac{dL}{du_1} + \frac{d}{dy} \frac{dL}{du_2}$$

2.2 Question 2

2.2.1 a)

En posant le Lagrangien de cette manière:

$$L(x, y, u(x, y), u_1(x, y), u_2(x, y)) = \|\nabla u(x, y)\|_2^2$$

Montrons que dans ce cas une solution u au problème de cette partie vérifie les conditions suivantes:

$$\begin{cases} \Delta u = 0 \\ u|_{d\Omega} = f|_{d\Omega} \end{cases}$$

On sait que la norme deux au carré du gradient de u s'écrit :

$$\|\nabla u(x, y)\|_2^2 = \left\| \begin{pmatrix} \frac{du}{dx} \\ \frac{du}{dy} \end{pmatrix} \right\|_2^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{du}{dy}\right)^2 = u_1^2 + u_2^2$$

On repart des équations d'Euler:

$$\begin{aligned} \frac{d}{du} &= \frac{d}{dx} \left(\frac{dL}{du_1} \right) + \left(\frac{d}{dy} \right) \left(\frac{dL}{du_2} \right) \\ \Leftrightarrow \frac{d}{du} (u_1^2 + u_2^2) &= \frac{d}{dx} \left(\frac{d}{du_1} (u_1^2 + u_2^2) \right) + \frac{d}{dy} \left(\frac{d}{du_2} (u_1^2 + u_2^2) \right) \\ \Leftrightarrow 0 &= \frac{d}{dx} (2u_1) + \frac{d}{dy} (2u_2) \\ \Leftrightarrow 0 &= 2 \frac{du_1}{dx} + 2 \frac{du_2}{dy} \\ \Leftrightarrow 0 &= \frac{d}{dx} \left(\frac{du_1}{dx} \right) + \frac{d}{dy} \left(\frac{du_2}{dy} \right) \\ \Leftrightarrow 0 &= \frac{d^2 u}{dx^2} + \frac{d^2 u}{dy^2} \\ \Delta u &= 0 \end{aligned}$$

De plus on sait que le bord n'est pas abimé donc dans ce cas: $u|_{d\Omega} = f|_{d\Omega}$

On a alors démontré qu'une solution u au problème de cette partie vérifie les conditions susmentionnées.

2.2.2 b)

Pour donner une justification du lagrangien précédent pour la résolution de notre problème de restauration d'image, il faut tout d'abord définir ce que représentent le Lagrangien et le gradient.

On sait tout d'abord que le Lagrangien représente les contraintes locales, et donc les espaces bruités de l'image. Il permet de résoudre un problème d'optimisation avec des contraintes, celui-ci se résout en cherchant des valeurs qui minimisent le Lagrangien. Le gradient, quant à lui, est une mesure de la variation locale de l'image. Sa norme en restauration d'image s'utilise pour mesurer la régularité. Lorsque le Lagrangien est égal à la norme du gradient, cela signifie que la régularisation basée sur le gradient est équilibrée avec la qualité de la reconstruction, de sorte que l'image restaurée est à la fois régulière et fidèle à l'image bruitée.

De plus nous pouvons faire une analogie physique pour mieux comprendre ce phénomène. Prenons par exemple une surface d'eau agitée, supposons que celle-ci représente l'image bruitée, et que nous voulons

la "restaurer" en obtenant une surface plane. Le Lagrangien peut être vu comme la tension de surface de l'eau, qui essaie de minimiser l'énergie de surface en ramenant la surface à un état d'équilibre stable. Le gradient, peut-être vu comme le vecteur normal à la surface de l'eau en tout point, qui représente la direction de la plus forte variation de la surface.

Une image restaurée, tout comme la surface d'eau, peut atteindre un état d'équilibre stable qui est à la fois lisse et fidèle à la topographie sous-jacente.

2.2.3 c)

Afin d'implémenter et résoudre numériquement le problème nous allons le discrétiser.

i.

Grâce au développement de Taylor d'ordre 2, on a d'une part:

$$g(x+h) = g(x) + hg'(x) + \frac{h^2}{2}g''(x) + h^3o(h)$$

D'autre part :

$$g(x-h) = g(x) - hg'(x) + \frac{h^2}{2}g''(x) + h^2o(x)$$

En additionnant les deux équations précédentes, on obtient :

$$g(x+h) + g(x-h) = 2g(x) + hg'(x) - hg'(x) + h^2g''(x) + 2h^2o(h)$$

$$\Leftrightarrow \frac{g(x+h) - 2g(x) + g(x-h)}{h^2} \approx g''(x)$$

quand $h > 0$ "petit".

ii.

Pour une image on supposera que le point $(x, y) \in \Omega$ est la position (x, y) d'un pixel et x est le numéro de ligne de la matrice et y le numéro de la colonne. On considérera alors $h = 1$. D'après la question précédente, on avait (P) :

$$\begin{cases} \Delta u = 0 \\ u|_{\partial\Omega} = f|_{\partial\Omega} \end{cases}$$

$$\Delta u = \frac{d^2 u_1}{dx^2} + \frac{d^2 u_2}{dy^2}$$

Avec $h=1$:

$$\frac{d^2 u_1}{dx^2} = u_{1x+1,y} - 2u_{1x,y} + u_{1x-1,y}$$

$$\frac{d^2 u_2}{dy^2} = u_{2x,y+1} - 2u_{2x,y} + u_{2x,y-1}$$

2.2.4 d)

F représente l'image abimée, U représente l'image avec les pixels noirs, et M représente masque.

On cherche à minimiser $J(U)$

$$J(U) = \frac{1}{2}(\|D_p U\|_F^2 + \|U D_q^T\|_F^2)$$

Comme on nous sommes dans une problème de minimisation, on peut enlever les carrés et le $\frac{1}{2}$ car minimiser $\operatorname{argmin} \frac{1}{2} \|x\|^2 = \operatorname{argmin} \|x\|^2 = 0$

Ainsi :

$$\operatorname{argmin} \frac{1}{2}(\|D_p U\|_F^2 + \|U D_q^T\|_F^2) = \operatorname{argmin}(\|D_p U\|_F + \|U D_q^T\|_F)$$

Or, grâce à inégalité triangulaire on a:

$$\operatorname{argmin}(\|D_p U\|_F + \|U D_q^T\|_F) \geq \operatorname{argmin}(\|D_p U + U D_q^T\|_F)$$

D'un autre côté, en partant du problème $\mathbb{P}$ on a:

$$D_p U + U D_q^T = 0 \Leftrightarrow \|D_p U + U D_q^T\|_F = 0 \Leftrightarrow \operatorname{argmin}(\|D_p U + U D_q^T\|_F) = 0$$

Au final:

$$\operatorname{argmin}(J(U)) \geq \operatorname{argmin}(\mathbb{P})$$

Dans cette deuxième partie, nous allons nous intéresser à la résolution numérique du problème final de la partie précédente en commençant par des questions mathématiques puis en continuant avec une application sur python.

3.1 Question 1

L'opérateur sous python qui correspond au produit matriciel de Hamard est *.

3.2 Question 2

Reprenons l'expressions de $J(U)$ déterminé dans la partie précédente.

$$J(U) = \frac{1}{2}(\|D_p U\|_F^2 + \|U D_q^T\|_F^2) = \frac{1}{2}(Tr(U^T D_p^T D_p U) + Tr(D_q U^T U D_q^T))$$

On sait que :

$$U = X + F$$

On a donc:

$$J(U) = \frac{1}{2}(Tr((X + F)^T D_p^T D_p (X + F)) + Tr(D_q (X + F)^T (X + F) D_q^T))$$

$$J(U) = \frac{1}{2}(Tr(X^T D_p^T D_p X + X^T D_p^T D_p F + F^T D_p^T D_p X + F^T D_p^T D_p F) \\ + Tr(D_q X^T X D_q^T + D_q X^T F D_q^T + D_q F^T X D_q^T + D_q F^T F D_q^T))$$

$$J(U) = \frac{1}{2}(Tr(X^T D_p^T D_p X) + Tr(D_q X^T X D_q^T) + Tr(X^T D_p^T D_p F) + Tr(F^T D_p^T D_p X) \\ + Tr(D_q X^T F D_q^T) + Tr(D_q F^T X D_q^T) + Tr(F^T D_p^T D_p F) + Tr(D_q F^T F D_q^T))$$

$$\Phi(X) = \frac{1}{2}\langle X, A(X) \rangle + \langle B, X \rangle$$

$$\Phi(X) = \frac{1}{2}Tr(X^T (D_p^T D_p X + X D_q^T D_q)) + Tr((F^T D_p^T D_p + D_q^T D_q F^T)X)$$

$$\Phi(X) = \frac{1}{2}Tr(X^T D_p^T D_p X) + Tr(X^T X D_q^T D_q) + Tr(F^T D_p^T D_p X) + Tr(D_q^T D_q F^T X)$$

Par identification:

$$J(U) = \frac{1}{2} \langle X, A(X) \rangle + \frac{1}{2} (2(\text{Tr}(X^T D_p^T D_p X) + 2\text{Tr}(D_q X^T X D_q^T) + \frac{1}{2}(\text{Tr}(F^T D_p^T D_p F) + \text{Tr}(D_q F^T F D_q^T)))$$

$$J(U) = \frac{1}{2} \langle X, A(X) \rangle + \langle B, X \rangle$$

On peut supprimer les deux dernières valeurs en tant que constantes. En effet on veut l'argument minimum de $J(U)$, et ce dernier reste inchangé avec ou sans ces deux dernières constantes.

$$\bar{M} = 1 - m_{ij}$$

si $X \in C$, $U - F \in C$ aussi. Or, $m_{ij} = 1, u_{ij} = f_{ij}$ quand le pixel est transmis. Nous savons que $X = U - F$, donc $X_{ij} = 0$ quand le pixel est transmis, ainsi afin de respecter la contrainte nous devons multiplier par $\bar{M}$ les termes.

3.3 Question 3

Dans cette question, on souhaite démontrer que l'opérateur $A(\bullet)$ est linéaire. On doit ainsi vérifier que $\forall X, Y$ et $\forall \alpha \in \mathbb{R}$, les conditions suivantes sont satisfaites :

$$A(X + Y) = A(X) + A(Y) \quad (\text{additivité})$$

$$A(\alpha X) = \alpha A(X) \quad (\text{homogénéité})$$

Commençons par vérifier la première condition :

$$\begin{aligned} A(X + Y) &= (D_p^T D_p (X + Y) + (X + Y) D_q^T D_q) \odot \bar{M} \\ A(X + Y) &= (D_p^T D_p X + D_p^T D_p Y + X D_q^T D_q + Y D_q^T D_q) \odot \bar{M} \\ A(X + Y) &= (D_p^T D_p X + X D_q^T D_q) \odot \bar{M} + (D_p^T D_p Y + Y D_q^T D_q) \odot \bar{M} \\ A(X + Y) &= A(X) + A(Y) \end{aligned}$$

La première condition est bien vérifiée.

Vérifions maintenant la deuxième condition :

$$\begin{aligned} A(\alpha X) &= (D_p^T D_p (\alpha X) + (\alpha X) D_q^T D_q) \odot \bar{M} \\ A(\alpha X) &= \alpha (D_p^T D_p X + X D_q^T D_q) \odot \bar{M} \\ A(\alpha X) &= \alpha A(X) \end{aligned}$$

La deuxième condition est bien vérifiée, de plus $A(0)=0$. Ainsi l'opérateur $A(\bullet)$ est bien linéaire.

3.4 Question 4

Pour montrer que pour A, B et C trois matrices de même taille $p \times q$ ont les mêmes les éléments diagonaux, il faut retourner à l'expression du produit matriciel en utilisant les sommes.

$$(C^T (A \odot B))_{ii} = \sum_i^q \sum_j^p c_{ij} a_{ij} b_{ij} = \sum_i^q \sum_j^p a_{ij} c_{ij} b_{ij} = (A^T (C \odot B))_{ii}$$

On doit également montrer que l'opérateur $A(\cdot)$ est symétrique pour le produit scalaire canonique, on a ainsi $\forall (X, Y) \in M_{p,q}(\mathbb{R})^2$:

$$\langle Y, A(X) \rangle = \text{tr}(Y^T A(X)) = \text{tr}((A(X))^T Y)$$

Or on sait d'après la question 3 que:

$$A(X) = (D_p^T D_p X + X D_q^T D_q) \odot \bar{M}$$

$$\Rightarrow \text{tr}((A(X))^T Y) = \text{tr}((D_p^T D_p X + X D_q^T D_q) \odot \bar{M})^T Y)$$

$$\Leftrightarrow \text{tr}((D_p D_p^T X^T + X^T D_q D_q^T) \odot \bar{M}^T) Y) = \text{tr}(X^T (D_p D_p^T Y + Y D_q D_q^T) \odot \bar{M}^T)$$

$$\Leftrightarrow \text{tr}(X^T A(Y)^T) = \langle A(Y), X \rangle$$

Ainsi l'opérateur $A(\bullet)$ est bien symétrique.

3.5 Question 5

$$D_H \Phi(X) = \lim_{\lambda \rightarrow 0} \frac{\Phi(X + \lambda H) - \Phi(X)}{\lambda}$$

$$\Leftrightarrow \lim_{\lambda \rightarrow 0} \frac{1}{\lambda} \left(\frac{1}{2} \langle X + \lambda H, A(X + \lambda H) \rangle + \langle B, X + \lambda H \rangle - \frac{1}{2} \langle X, A(X) \rangle - \langle B, X \rangle \right)$$

$$\Leftrightarrow \lim_{\lambda \rightarrow 0} \frac{1}{\lambda} \left(\frac{1}{2} \text{Tr}((X + \lambda H)^T A(X + \lambda H)) + \text{Tr}(B^T (X + \lambda H)) - \frac{1}{2} \text{Tr}(X^T A(X)) - \text{Tr}(B^T X) \right)$$

$$\Leftrightarrow \lim_{\lambda \rightarrow 0} \frac{1}{\lambda} \left(\frac{1}{2} \text{Tr}(X^T A(X + \lambda H) + H^T \lambda A(X + \lambda H)) + \text{Tr}(B^T X + B^T \lambda H) - \frac{1}{2} \text{Tr}(X^T A(X)) - \text{Tr}(B^T X) \right)$$

$$\Leftrightarrow \lim_{\lambda \rightarrow 0} \frac{1}{\lambda} \left(\frac{1}{2} \cancel{\text{Tr}(X^T A(X))} + \frac{1}{2} (X^T A(\lambda H)) + \frac{1}{2} \text{Tr}(H^T \lambda A(X)) + \frac{1}{2} \text{Tr}(H^T \lambda A(\lambda H)) \right)$$

$$+ \cancel{\text{Tr}(B^T X)} + \text{Tr}(B^T \lambda H) - \frac{1}{2} \cancel{\text{Tr}(X^T A(X))} - \cancel{\text{Tr}(B^T X)})$$

$$\Leftrightarrow \lim_{\lambda \rightarrow 0} \frac{1}{2} \text{Tr}(X^T A(H)) + \frac{1}{2} \text{Tr}(H^T A(X)) + \frac{1}{2} \lambda \text{Tr}(H^T A(H)) + \text{Tr}(B^T H)$$

$$\Leftrightarrow \frac{1}{2} \text{Tr}(X^T A(H)) + \frac{1}{2} \text{Tr}(H^T A(X)) + \text{Tr}(B^T H)$$

On sait aussi que: $\text{Tr}(X^T A(H)) = \langle X, A(H) \rangle = \langle A(X), H \rangle = \text{Tr}(A(X)^T H) = \text{Tr}(H^T A(X))$

On a donc: $D_H \Phi(X) = \text{Tr}(X^T A(H)) + \text{Tr}(B^T H) \Leftrightarrow \text{Tr}(H^T (A(X) + B)) \Leftrightarrow \langle H, A(X) + B \rangle$

On peut donc en conclure que le gradient est $A(X) + B$.

3.6 Question 6

Le défi principal de ce projet est de traduire l'algorithme du gradient conjugué qu'on a vu en TD et qui s'écrit :

$$\left\{ \begin{array}{l} x_0 \in \mathbb{R}^n \\ d_0 = -\nabla f(x_0) \\ \text{Pour } k \geq 0 \\ t_r = \frac{d_k^T \nabla f(x_k)}{d_k^T A d_k} \\ x_{k+1} = x_k + t_k d_k \\ \beta_k = \frac{d_k^T A \nabla f(x_{k+1})}{d_k^T A d_k} \\ d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k \end{array} \right.$$

en sa version matricielle.

Pour se faire on remplace le produit scalaire de vecteurs $\langle x, y \rangle = x^T y$ par le produit scalaire matriciel $\langle A, B \rangle = \text{Tr}(A^T B)$.

$$\left\{ \begin{array}{l} d_0 = -\nabla f(x_0) \\ \text{Pour } k \geq 0 \\ t_r = \frac{-\langle d_k, \nabla f(x_k) \rangle}{\langle d_k, A(d_k) \rangle} \\ x_{k+1} = x_k + t_k d_k \\ \beta_k = \frac{-\langle d_k, A(\nabla f(x_{k+1})) \rangle}{\langle d_k, A(d_k) \rangle} \\ d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k \end{array} \right.$$

3.7 Question 7

3.7.1 a) masque(F,v)

Nous allons ici programmer une fonction masque(F,v) qui prend en entrée une matrice F représentant une image avec des pixels noirs que nous avons rajoutés sur la photo originale. De plus on utilise également v un entier proche de 0 et qui renvoie en sortie la matrice du masque.

Programme de la fonction masque(F,v):

```
1
2  ### Fonction masque pour les image en noir et blanc
3
4  def masque(F,v):
5      return np.where(F<=v,0,1)
6
7  ### Fonction masque pour les images en couleur
8
9  def masque_couleur(F,v):
10     M=np.zeros(np.shape(F)[0:2])
11     for i in range(np.shape(F)[0]):
12         for j in range(np.shape(F)[1]):
13             if F[i,j,0]<=v and F[i,j,1]<=v and F[i,j,2]<=v:
14                 M[i,j]=0
15             else:
16                 M[i,j]=1
17     return M
18
19 def masque_couleur2(F,v):
20     m1=masque(F[:, :, 0],v)
21     m2=masque(F[:, :, 1],v)
22     m3=masque(F[:, :, 2],v)
23     return m1*m2*m3
```

Nous avons définis une fonction masque pour les images en noir et blanc afin de limiter un problème de groupement de pixels bleue qui apparaissent lors de la restauration de l'image test-2. Ce problème sera traité dans la partie difficulté.

3.7.2 b) D(r)

Nous allons ici programmer une fonction D(r) qui prend en entrée un entier et qui renvoie la matrice Dr $M_{r-2,r}(\mathbb{R})$ de l'énoncé.

Programme de la fonction D(r):

```
1  def D(r):
2      D=np.diag(np.ones(r))-2*np.diag(np.ones(r-1),1)+np.diag(np.ones(r-2),2)
3      return D[0:r-2,:]
```

On utilise la fonction diag de numpy afin d'éviter de créer terme par terme les composantes de la matrice D (avec une double boucle for i) et ainsi réduire le temps de calcul.

On décale les diagonales de 1 et de 2 vers le haut (avec le 2eme argument de la fonction diag) pour former la matrice triangulaire supérieure D de l'énoncé

3.7.3 c) $A(X,M)$

Nous allons ici programmer une fonction $A(X,M)$ qui prend en entrée une matrice X et la matrice du masque M et qui renvoie l'opérateur $A(X)$ de l'énoncé.

Programme de la fonction $A(X;M)$:

```
1 def A(X, M):
2     p,q = np.shape(X)
3
4     Dp=D(p)
5     Dq=D(q)
6
7     A = (Dp.T@Dp@X + X@Dq.T@Dq) * ((M-1)*(-1))
8     return A
```

3.7.4 d) $GCM(M,F,X0,epsilon)$

Nous allons ici programmer une fonction implémentant l'algorithme du gradient conjugué matriciel $GCM(M,F,X0,epsilon)$ où M est la matrice du masque, F la matrice de l'image abîmée, $X0$ une matrice d'initialisation et un réel représentant une précision souhaitée. On pourra prendre dans la pratique $epsilon = 1$.

Programme de la fonction $GCM(M;F;X0;epsilon)$:

```
1 def GCM(M,F,X0,epsilon=1):
2     AFM = A(F,M)
3
4     img=[]
5
6     x=copy.copy(X0)
7     d=-A(x,M)-AFM
8     compteur=0
9     y=x+epsilon*np.ones(np.shape(x))
10    while np.linalg.norm(x-y)>epsilon and compteur<200:
11        y=np.copy(x)
12
13        AdM = A(d,M)
14
15        t=-(np.trace(d.T@(A(x,M)+AFM)))/(np.trace(d.T@AdM))
16        x=x+t*d
17
18        AxM = A(x,M)
19
20        beta=(np.trace(d.T@A((AxM+AFM),M)))/(np.trace(d.T@AdM))
21        d=(-AxM-AFM)+beta*d
22        compteur+=1
23        print("iteration {}: {}".format(compteur))
24
25        img.append(x+F)
26
27    return x,img
```

3.7.5 e) restauration(F,M,epsilon)

Nous allons ici programmer une fonction restauration(F, M, epsilon) qui utilise les fonctions précédentes et qui renvoie une image restaurée. Pour ce faire, nous appliquons l'algorithme du gradient conjugué matriciel aux trois couches de l'image en couleur (ou à la seule couche de l'image en noir en blanc). Cet algorithme permet de "restaurer" les pixels noirs. Il nous suffit alors dans les fonctions restauration d'additionner les matrices x que renvoie la fonction GCM, qui représente la dernière étape de la restauration, et la matrice F qui représente l'image "abîmée" (avec les pixels noirs).

On a également inséré dans les fonctions restauration tout le code nécessaire pour réaliser une vidéo des étapes de résolution de l'algorithme.

Programme de la fonction restauration(F;M;epsilon):

```
1 def restauration(F,M,epsilon=1):
2     X0=np.zeros(np.shape(F))
3     x,img=GCM(M,F,X0,epsilon)
4     #On definit les parametres de la video
5     fps = 20
6     hauteur,longueur = np.shape(F)[0],np.shape(F)[1]
7     fourcc = cv2.VideoWriter_fourcc(*"mp4v")
8     video = cv2.VideoWriter("sortie_video.mp4", fourcc, int(fps), (longueur,
9         hauteur))
10
11     for i in range(len(img)):
12         cv2.imwrite("lesmathcestbien.png",np.uint8(img[i]))
13         im = cv2.imread("lesmathcestbien.png")
14         # On ajoute chaque image (matrice) a la video
15         video.write(im)
16         os.remove("lesmathcestbien.png")
17     video.release()
18     return np.uint8(x+F)
19
20 %% Restauration couleur
21
22 def restauration_couleur(F,M,epsilon=1):
23
24     xfinal=np.zeros(np.shape(F))
25     X0=np.zeros(np.shape(F[:, :, 0]))
26     #On definit les parametres de la video
27     fps = 20
28     hauteur,longueur = np.shape(F)[0],np.shape(F)[1]
29     fourcc = cv2.VideoWriter_fourcc(*"mp4v")
30     video = cv2.VideoWriter("sortie_video.mp4", fourcc, int(fps), (longueur,
31         hauteur))
32
33     xr,imgr=GCM(M,F[:, :, 0],X0,epsilon)
34     xfinal[:, :, 0]=xr
35
36     xv,imgv=GCM(M,F[:, :, 1],X0,epsilon)
37     xfinal[:, :, 1]=xv
38
39     xb,imgb=GCM(M,F[:, :, 2],X0,epsilon)
40     xfinal[:, :, 2]=xb
41
42     I=np.zeros(np.shape(F))
43     for i in range(len(imgr)):
44         I[:, :, 0]=imgr[i]
45         I[:, :, 1]=imgv[i]
46         I[:, :, 2]=imgb[i]
47
48     video.write(np.uint8(I))
49     video.release()
50     return np.uint8(xfinal+F)
```

3.7.6 Difficultés en python

Lors de la réalisation du programme python, nous ne sommes pas arrivé à résoudre un problème du groupement de pixels bleus au moment de la restauration de l'image test-2. En effet, même après un très grand nombre d'itérations, les pixels bleus n'arrivent pas à disparaître. Le même problème à très moindre échelle est aussi présent sur la veste de l'homme sur la photo orig1. La même difficulté est présente dans la photo test-1 et test-2 mais cette fois si avec des groupements de pixels blancs.

Ces problème peuvent s'expliquer de plusieurs manières différentes. La première explication serait que l'algorithme du gradient conjugué matriciel que nous avons codé ne converge pas en ces points là pour une raison que nous ignorons. La deuxième explication serait que ces problème peuvent provenir des masques que nous avons fait des images en couleur et en noir et blanc. C'est pour cela nous avons défini une fonction masque-couleur pour les images en couleur qui nous permet de minimiser ce problème de point bleue.

Après la restauration on obtient ainsi les images suivantes avec encore quelques défauts mais on peut observer un progrès notable
Voici le changement pour l'image orig1 sur laquelle on ne voit plus les défauts liés à l'usure et au temps.}



Figure 3.1: Orig1 avant restauration



Figure 3.2: Orig1 après restauration

Nous avons également fais la restauration de Orig1 en noire et blanc et cela nous donne:



Figure 3.3: Orig1 avant restauration



Figure 3.4: Orig1 après restauration (300 itérations)

Ensuite pour l'image test1 il fallait restaurer un grand nombre de pixels noirs pour voir apparaitre la photo d'une femme. On obtient ainsi:

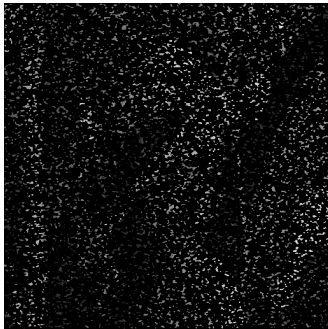


Figure 3.5: Test1 avant restauration

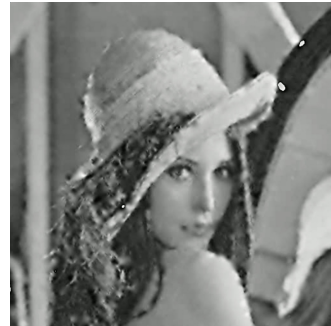


Figure 3.6: Test1 après restauration (500 itérations)

Nous avons également fais la restauration de Test2 en noir et blanc et cela nous donne:



Figure 3.7: Test2 en noir et blanc avant restauration



Figure 3.8: Test2 en noir et blanc après restauration(350 itérations)

Enfin la dernière image était test2 représentant des fraises. On obtient par le même cheminement que précédemment la restauration suivante:

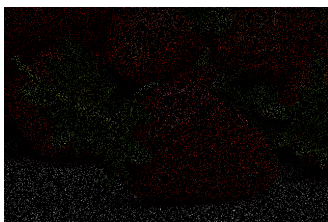


Figure 3.9: Test2 avant restauration

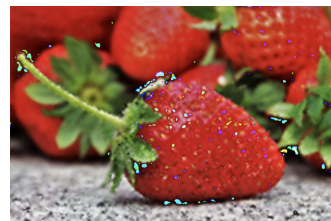


Figure 3.10: Test2 après restauration (400 itérations)

Pour aller plus loin dans l'étude de ce BE nous nous sommes amusés à prendre des photos plus ou moins personnelles pour tester. Voici le père de nathan:



Figure 3.11: Le père de nathan avant restauration



Figure 3.12: Le père de nathan après restauration (500 itérations)

Ensuite nous nous sommes souvenus que M. Couffignal nous avait parlé d'images qui pouvaient se propager comme des ondes à la surface de l'eau nous avons donc testé et obtenu le résultat suivant:

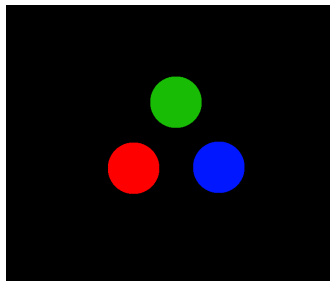


Figure 3.13: Les couleurs primaires avant restauration

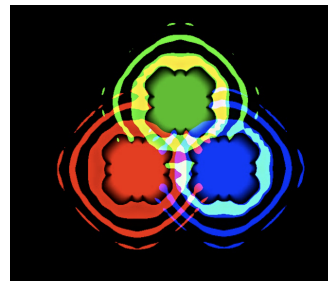


Figure 3.14: Les couleurs primaires à l'étape 500

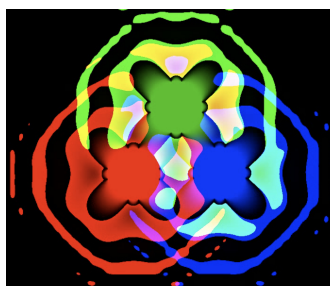


Figure 3.15: Les couleurs primaires à l'étape 1000

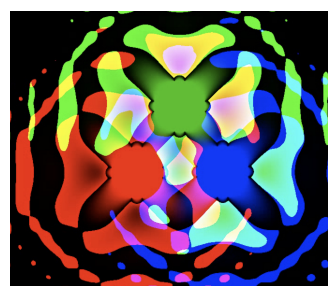


Figure 3.16: Les couleurs primaires à l'étape 2000

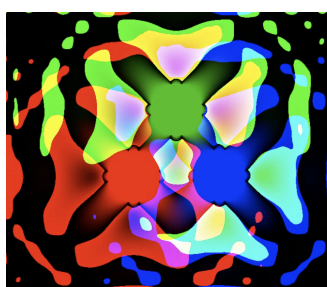


Figure 3.17: Les couleurs primaires après 5000 itérations

On remarque que les trois ronds de couleurs primaires (rouge, vert et bleue) se propagent de façon circulaire, comme un caillou qu'on jèterais 'a la surface de l'eau. Lorsque les ondes de couleurs se superposent, elles forment de nouvelle couleur, on retrouve ainsi le principe de synthèse additive des couleurs vue au col'Ege : rouge + blue = magenta, rouge + vert = jaune, vert + bleue = cyan et enfin rouge + bleue + vert = blanc, qu'on peut retrouver en certain point de l'image.

Afin de conclure ce rapport, nous souhaitons tout d'abord remercier nos professeurs de mathématiques M. EL MAHBOUBY et M. COUFFIGNAL pour le temps qu'ils ont consacré à nous aider dans la compréhension de ce BE.

Ce bureau d'étude a été pour nous l'occasion de travailler en équipe. Nous avons fait face tous ensemble aux différentes difficultés rencontrées.

Une difficulté majeure que nous avons rencontrée a été de faire le lien entre la partie théorique et la restauration d'images sur python. Il a été de même difficile d'interpréter et donner un sens physique aux résultats trouvés, ce qui nous a demandé une grande quantité de temps, mais aussi d'une réflexion plus poussée que simplement trouver le résultat et le commenter.

Pour étayer notre propos nous avons parfois eu des problèmes de restauration des images ce qui a entraîné de nombreuses modifications de notre code python, notamment la fonction GCM et restauration. En effet nous avons des sorties d'images avec seulement les couleurs rouge bleu et vert, sans amélioration de la qualité de l'image.

Ce projet nous a permis de redécouvrir le fonctionnement du gradient conjugué matriciel ainsi que ses nombreuses propriétés et applications dans le monde moderne. Mais il nous a également permis de consolider nos savoirs sur le module lié à l'optimisation différentiable.